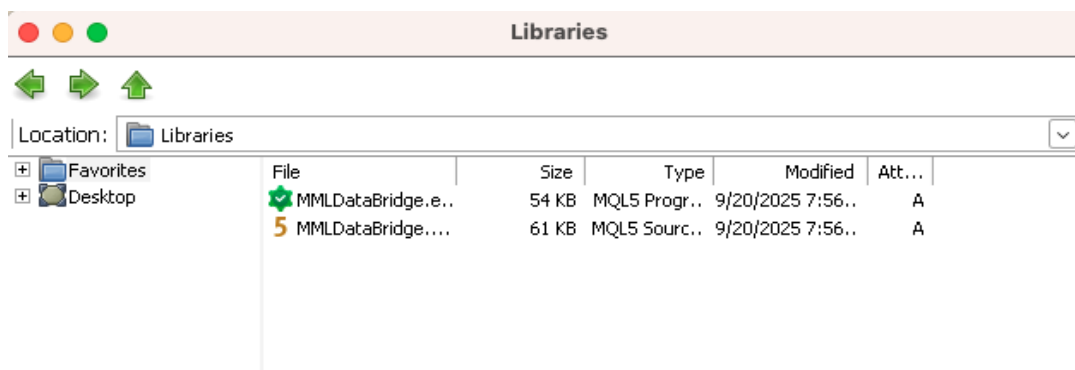


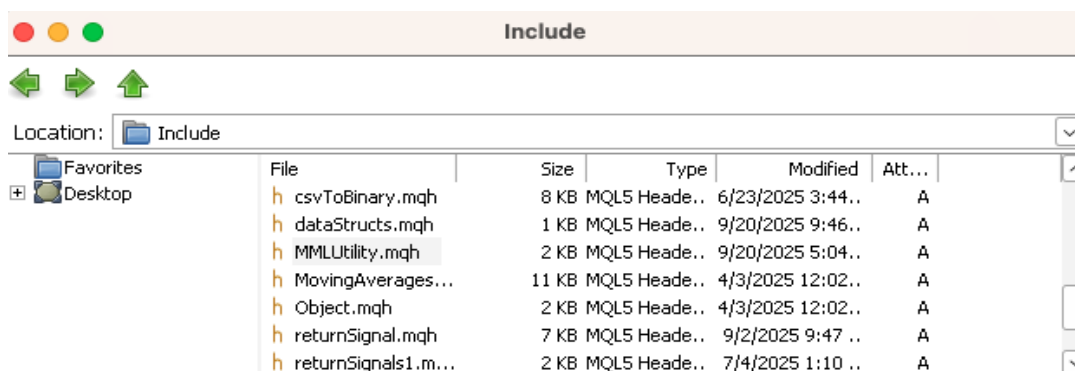
MML Bridge User Set Up Manual

The purpose of this program is to return machine learning predictions into the Meta Trader 5 trading platform. Meta ML Bridge is therefore a developers toolkit to simplify loading data into MT5 for backtesting, optimization and live trading. Meta ML Bridge is also designed for developers to quickly and easily verify the integrity of data ingested into MT5. The program can be set up in 8 easy steps

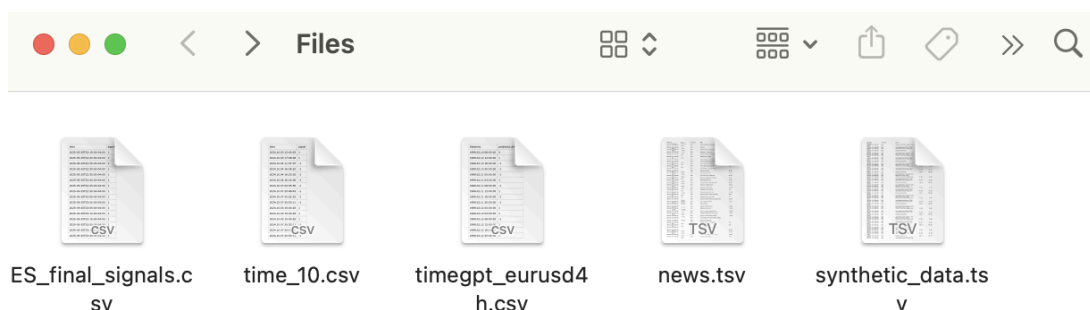
- ❖ 1. Move the executable code (MMLDataBridge.exe) into the library directory on mt5



- ❖ 2. Move MMLUtility.mqh to MT5's include directory. This file includes wrapper functions that
 - calls the libraries exported functions. Create a dataStructs.mqh file we will use later to define our data with.



- ❖ 3. Move data to be ingested into MT5's common files directory



- ❖ 4. Create an EA you would like to bridge external csv/tsv data with. The screenshot below shows the standard way to initialize the data pipeline across all expert advisors.

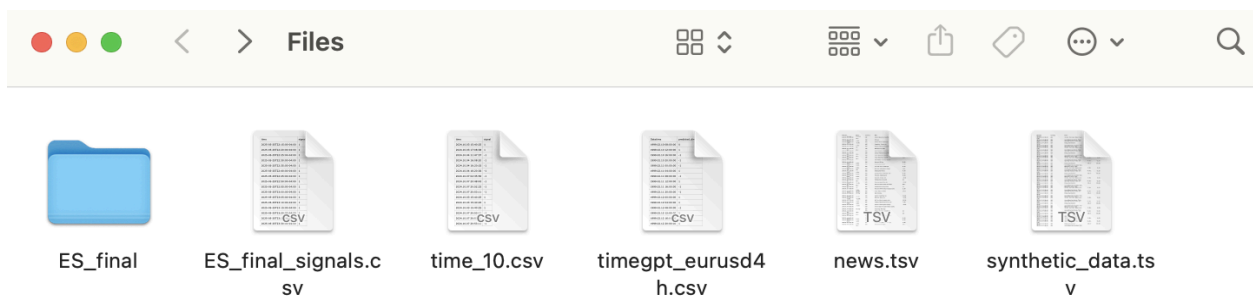
- Include the <MMLUtility.mqh> file
 - All wrapper functions are listed here
- Include a <dataStructs.mqh> file
 - Store data structs
- Define the templated bool returnData("filename", dataStruct) function
 - Arguments: The name of a file to return data from, a data struct to populate data
- Call initializeBridge("EAname") inside OnInit()
 - Arguments: A name for the EA file bridge created inside MT5's common files
- Call shutDownBridge() inside OnDeinit

```
1 #property strict
2 #include <Trade/Trade.mqh>
3 #include <MMLUtility.mqh>
4 #include <dataStructs.mqh>
5
6 CTrade trade; // trade object
7
8 // ----- bridge decloration -----
9 template<typename T>
10 bool returnData(const string filename, T &out);
11
12 //+-----+
13 //| Expert initialization |
14 //+-----+
15 int OnInit() {
16     initializeBridge("ES_final");
17     return INIT_SUCCEEDED;
18 }
19
20 //+-----+
21 //| Expert deinitialization |
22 //+-----+
23 void OnDeinit(const int reason) {
24     shutDownBridge();
25 }
26
27 //+-----+
28 //| Expert tick |
29 //+-----+
30 void OnTick() {
31
32 }
33
```

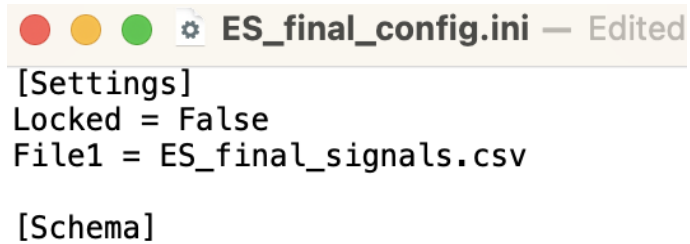
- ❖ 4.1. A configuration file is used to list the data being ingested into MT5. The user has the option of creating the config.ini file manually or running the above program. Running the above program will create the required folder and config.ini file inside MT5's common files directory. Running the program for the first time without the correct folder/file will give the following messages in the backtester. These messages indicate the config file has been created successfully and we are now ready to input data.

- 2025.05.24 00:00:00 Initializing bridge: EA Name = 'ES_final'
- 2025.05.24 00:00:00 Config file not found — creating new template: ES_final\ES_final_config.ini
- 2025.05.24 00:00:00 Created a new template. Please add input file names and rerun.
- 2025.05.24 00:00:00 Schemas are NOT locked
- 2025.05.24 00:00:00 Invalid input: Please ensure 'Example1.csv' is placed in common files directory
- 2025.05.24 00:00:00 Parsed 0 file(s) from config.

- ❖ 4.2 For manual setup, create a folder in MT5's common files directory with the same EA name passed to InitializeBridge(). Create a config.ini file named EAname_config.ini inside the folder



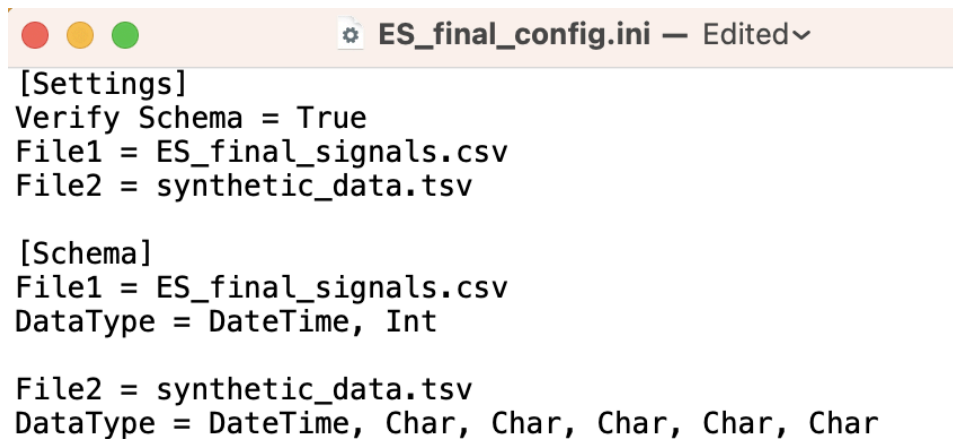
- ❖ 5. Open the config.ini file, and input data you would like to use. Acceptable data includes csv or tsv files.

A screenshot of a text editor window titled "ES_final_config.ini — Edited". The window contains the following text:

```
[Settings]
Locked = False
File1 = ES_final_signals.csv

[Schema]
```

- ❖ 5.1. When inputs are added to the config.ini file, the next program run will scan all data in the files and generate a data type schema. Note that some IDE or text editors will automatically refresh while others may require the file to be closed and reopened.

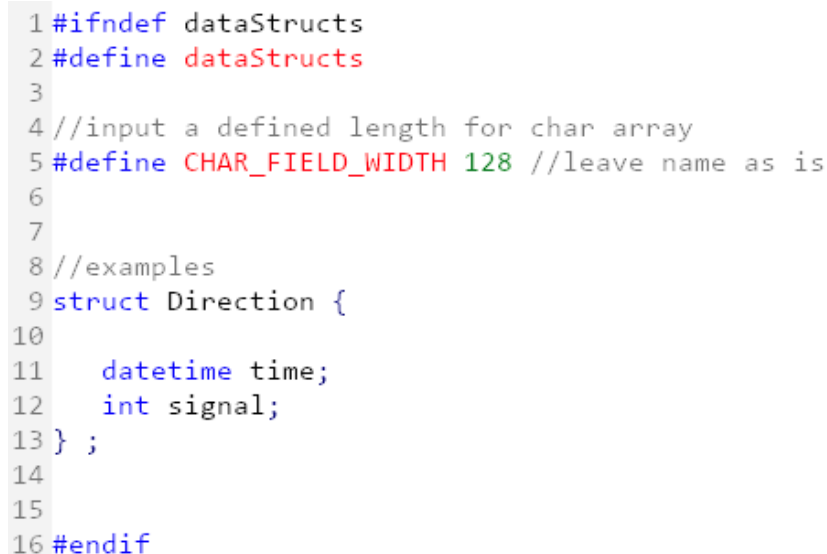
A screenshot of a text editor window titled "ES_final_config.ini — Edited". The window contains the following text:

```
[Settings]
Verify Schema = True
File1 = ES_final_signals.csv
File2 = synthetic_data.tsv

[Schema]
File1 = ES_final_signals.csv
DataType = DateTime, Int

File2 = synthetic_data.tsv
DataType = DateTime, Char, Char, Char, Char
```

- ❖ 6. Use this schema to define a data struct inside your dataStruct.mqh file previously created in MT5's include directory.

A screenshot of a code editor showing the definition of dataStructs in a file named dataStruct.mqh. The code is as follows:

```
1 #ifndef dataStructs
2 #define dataStructs
3
4 //input a defined length for char array
5 #define CHAR_FIELD_WIDTH 128 //leave name as is
6
7
8 //examples
9 struct Direction {
10
11     datetime time;
12     int signal;
13 } ;
14
15
16 #endif
```

- ❖ 7. Declare an instance of the object above the OnTick() function. Call the function by passing the name of the file to read data from and the object to populate the data with. Below is an example of overloading the returnData() to populate different structs from different files.

```
27 //Declare data structures
28 Direction direction;
29 News news;
30
31 //+-----+
32 //| Expert tick |
33 //+-----+
34 void OnTick() {
35
36     // populates the direction data struct
37     if(returnData<Direction>("ES_final_signals.csv", direction)) {
38         //Add trade logic here
39         Print(direction.time, " ", direction.signal);
40     }
41
42     if(returnData<News>("synthetic_data.tsv", news)) {
43
44         Print(news.time, " ",
45             CharArrayToStr(news.currency), " ",
46             CharArrayToStr(news.event), " ",
47             CharArrayToStr(news.actual), " ",
48             CharArrayToStr(news.previous), " ",
49             CharArrayToStr(news.forecast));
50     }
51
52 }
```

❖ 8. Develop, trade and test your machine learning predictions

